

Orchestrating an Interpreter: Csound7 as a Host for Symbolic Programs

Hlöðver Sigurðsson

Independent
hlolli@gmail.com

Abstract. This paper evaluates Csound7’s Orchestra language as a host for complex symbolic programs. As a case study, I implemented a Clojure-inspired Lisp interpreter entirely in Orchestra using the Make-A-Lisp process. The interpreter uses recursive structs, arrays, and typed user-defined opcodes to implement lexical environments, closures, macros, tail calls, and exceptions. A musical example uses first-class graph values to build instruments and schedule musical patterns, keeping the symbolic program and audio engine in one process. The work exposed defects in type propagation, nested struct arrays, overload selection, pass-by-reference semantics, recursion safety, and default initialization, leading to fixes merged into Csound7’s development branch. The main limitation in the version studied here was memory lifetime: temporary recursive values persisted until the enclosing instrument ended, making even a small self-hosting test costly in memory. The result shows that Csound7 can support substantial symbolic programs close to its music engine, and that the lifetime of structured values matters for programs that allocate heavily.

Keywords: Csound7, language implementation, recursive data, user-defined opcodes, live coding

1 Introduction

Csound has been my main environment for making music and building the tools around it. As those tools grew, I kept asking more of the Orchestra language. This project began with a simple question: how far can that language be taken before another runtime has to take over?

An interpreter is a useful way to explore that question. It combines recursive data, parsing, name lookup, functions, mutable state, errors, and interaction in one relatively small program. If these parts compose cleanly, Csound gains more than an interpreter. The same facilities become available to musical tools that need structured state before or alongside signal processing.

Work began in 2022, when Csound7 structs made it possible to represent an abstract syntax tree directly. The first attempt was paused because recursion, nested arrays, and forward declarations were not yet dependable enough for a clear implementation. Once those features matured, the same project became a way to test them together rather than one at a time.

The language used here is MAL, a small Clojure-inspired Lisp with an incremental implementation guide and a shared conformance suite [2,7]. The upstream suite supplies the cases and expected results, so the conformance criterion is independent of this implementation. MAL provides a broad workload ending in the demanding test of running its own interpreter. The interpreter source code and examples are publicly available [8].

2 A Symbolic Program in Orchestra

2.1 Recursive Values

The most important Csound7 feature in this work is a user-defined type that can contain an array of its own type [1].

The central MAL value is therefore an ordinary recursive Orchestra value, shown here without book-keeping fields:

```
struct MalValue type:i, number:i, string:S, \  
  list:MalValue[], env:MalEnv[], metadata:MalValue[]
```

The `list` field holds syntax trees, lists, vectors, maps, function definitions, and error payloads. A type tag says which fields are meaningful. Unlike a tagged union, this representation keeps all fields present, so the interpreter must check which ones apply. It keeps UDO signatures simple and makes the tree readable without parallel tables or encoded indices.

The reader returns a value together with its new position. This small detail is important because returning only a `MalValue` worked for flat forms, but nested forms also need to tell their caller how much input they consumed. Lists, vectors, and maps then share one delimiter-driven reader, while reader macros such as quotation are translated into ordinary MAL forms.

Optional fields such as metadata and the outer environment use empty arrays when absent and one-element arrays when present. Atoms, closures, interned syntax trees, and lexical environments use integer handles into typed global registries so references can share the same value.

2.2 Evaluation with UDOs

New-style UDOs provide typed inputs, typed outputs, local variables, recursion, and pass-by-reference behavior [3]. They map naturally to the evaluator’s small operations: look up a symbol, evaluate a collection, apply a function, or return an error. Forward declarations allow the mutually recursive evaluator helpers to be checked before their bodies appear.

Operators are ordinary values in the environment. A MAL form such as `(+ 2 3)` therefore follows the same lookup and application path as a user function. Listing 1 condenses the relevant Orchestra code. Numeric arguments are calculated immediately. If either argument is a Csound graph node, the same operator constructs another node for later compilation.

```
env = MalEnvSet(env, "+", MalMkBuiltinOperator("+"))

opcode MalApplyBuiltinOperator(fn:MalValue, args:MalValue):MalValue
  result:MalValue = MalMkValue($MAL_NUMBER_TYPE)
  left:MalValue = MalAt(args, 0)
  right:MalValue = MalAt(args, 1)
  if (MalIsCsoundNode(left) || MalIsCsoundNode(right)) then
    result = MalMkCsoundInfix(fn.string, args)
  else
    result = MalMkNumber(left.number + right.number)
  endif
  xout result
endop
```

Listing 1. Condensed Orchestra code for applying the MAL addition operator to numbers or Csound graph values. Argument-count and type checks are omitted.

Tail calls are handled by keeping a working syntax tree and environment inside the evaluator loop. A selected `if` branch, a `let*` body, the last form of `do`, or a MAL function body replaces those working values and continues the loop. Argument evaluation remains recursive because it is not in tail position. This keeps the evaluator close to the language definition while allowing a 10,000-step tail-recursive call to complete without growing the Csound stack.

There is no host exception mechanism suitable for carrying an arbitrary MAL value. Errors are therefore values with a dedicated tag and optional payload. Every reader and evaluator boundary checks that tag before continuing. This is verbose, but predictable, and avoids unwinding Csound’s execution machinery.

3 What the Interpreter Asked of Csound7

The direct design mostly worked. The harder problems came from combinations that small examples rarely exercise. An array could belong to a recursive struct, be indexed inside a function argument, returned through a UDO, and then used in a condition. Several such cases lost type information, selected the wrong opcode overload, or crashed the compiler. Pass-by-reference UDOs could overwrite an aliased input before it was read. Infinite UDO recursion could exhaust the native stack. Empty arrays and default struct values were also difficult to express without arbitrary placeholders.

The resulting compiler and runtime fixes are merged into the official Csound7 development branch for inclusion in the Csound7 release [9]. A substantial program found gaps between features that already passed their isolated tests.

This process also changed the interpreter source. Early versions split direct member expressions into temporary variables to avoid compiler failures. Once the compiler was corrected, the clearer expressions could be restored. A workaround that compiles is not always evidence that the language abstraction works well. Returning to the direct form is part of the result.

4 Returning the Language to Music

A Lisp hosted by Csound should be useful for music without losing Csound’s portability. The core MAL environment therefore remains independent of the host, while a separate environment exposes explicitly qualified values such as `csound/pluck`, `csound/linseg`, and `csound/outs`. Applying these values constructs a typed Csound graph. Arithmetic on graph nodes constructs more graph expressions rather than prematurely calculating a number.

The `definst` macro turns such a graph into a Csound instrument and returns a callable MAL value. Calling it schedules an ordinary Csound event.

```
(definst harpsichord [note 60 amp 0.035]
  (let* [freq (csound/cpsmidinn note)
        signal (-> (csound/pluck 1 freq freq 0 1) (* amp))]
    (csound/outs signal signal)))

(def! notes {:C4 60 :D4 62 :E4 64 :F4 65 :G4 67 :C5 72})
;; Each entry is [beat pitch].
(def! melody
  [[0 :C4] [1 :C4] [2 :C4] [2.67 :D4] [3 :E4]
   [4 :E4] [4.67 :D4] [5 :E4] [5.67 :F4] [6 :G4]
   [8 :C5] [8.33 :C5] [8.66 :C5]
   [9 :G4] [9.33 :G4] [9.66 :G4]
   [10 :E4] [10.33 :E4] [10.66 :E4]
   [11 :C4] [11.33 :C4] [11.66 :C4]
   [12 :G4] [12.67 :F4] [13 :E4] [13.67 :D4] [14 :C4]])

(def! metro (partial * 0.2))
(def! after
  (fn* (beats clock) (comp clock (partial + beats))))
(def! play
  (fn* (clock melody)
    (mapv (fn* (entry)
              (csound/event "i" harpsichord (clock (first entry))
                            0.9 (get notes (second entry))))
          melody)))
(def! play-round
  (fn* (clock melody)
    (mapv (fn* (delay) (play (after delay clock) melody))
          [0 4 8 16])))

(play-round metro melody)
(csound/event "e" 7.2 0)
```

Listing 2. A simplified Csound adaptation of Ford’s Overtone canon, including the melody and note scheduling [4].

The instrument sends the same signal to both outputs. Each melody entry pairs a beat with a pitch name. The `metro` function converts beats to seconds, and `after` shifts that clock. The `play` function schedules each note as a Csound event. Finally, `play-round` starts the melody at four beat offsets. Overtone, created by Jeff Rose and Sam Aaron, provides a useful comparison because it connects Clojure to SuperCollider [5]. Here the symbolic language and audio engine share one Csound process.

To add the instrument to that process, `definst` converts its graph into Orchestra code and compiles it with `evalstr` [6]. The MAL evaluator handles the Lisp forms that build the graph and schedule its notes.

Interaction also has to respect Csound’s rate model. A terminal read must not block the performance thread, while the evaluator is easiest to run at init time. A non-blocking k-rate `readline` opcode therefore places completed lines in a bounded inbox, then triggers an init-time evaluation.

5 Results and the Remaining Boundary

The interpreter passed the required cases in MAL’s conformance suite and the Csound interoperability tests. The only mismatch was a soft `time-ms` check caused by a macOS clock bug, since fixed in Csound’s development branch [9].

Memory lifetime remained the harder limit. In the July version, the reference MAL interpreter could run a program that printed a single line, but sampled resident memory reached about 16 GiB.

Reduction pointed to the lifetime of temporary recursive structs. Allocations made in nested init-time UDO calls remained attached to the enclosing instrument instead of being reclaimed when a call returned. Registries, reusable environment slots, syntax interning, and shared empty arrays reduced duplication, but could not reclaim host temporaries whose lifetime the engine controlled.

This limit matters beyond Lisp. Score generators, analysis pipelines, and declarative instrument builders may all create short-lived structured values at init time. A deterministic call-frame or region lifetime would fit this case better than a tracing collector on the audio thread.

6 Conclusion

This project started from a desire to make Csound itself a better place for building musical tools. Implementing MAL showed that Csound7 can now express a surprisingly complete symbolic runtime in readable Orchestra code. It also turned difficult interpreter failures into small compiler and runtime fixes now merged upstream, covering UDOs, arrays, and recursion handling.

Programs that belong close to the music engine can now stay there for longer, run wherever Csound runs, and avoid an additional runtime. I hope that continuing to push Csound to its limits, and occasionally a little beyond them, will serve the same purpose: edge cases become visible and Csound becomes more dependable for work that nobody has tried yet.

References

1. Csound Developers: What’s New in Csound7.
<https://csound.com/manual/intro/whats-new-in-csound-7/> (accessed 12 September 2026)
2. Martin, J.: mal: Make a Lisp. Implementation guide, reference interpreters, and conformance tests. <https://github.com/kanaka/mal> (2014)
3. Csound Developers: opcode. Csound7 Manual. <https://csound.com/manual/opcodes/opcode/> (accessed 12 September 2026)
4. Ford, C.: Row, Row, Row Your Boat! Overtone example. <https://gist.github.com/ctford/2877443> (2012)
5. Aaron, S.: Overtone. In: Collaboration and Learning through Live Coding. Dagstuhl Reports 3(9), 152 (2014)
6. Csound Developers: evalstr. Csound7 Manual. <https://csound.com/manual/opcodes/evalstr/> (accessed 12 September 2026)
7. Hickey, R.: A History of Clojure. Proceedings of the ACM on Programming Languages 4(HOPL), Article 71, 1–46 (2020)
8. Sigurðsson, H.: csound-make-a-lisp. Source code and examples.
<https://github.com/hlolli/csound-make-a-lisp>
9. Csound Developers: Csound development source.
<https://github.com/csound/csound/tree/5f9b8a99da04c50baa739d016039f67886cd8146> (accessed 12 September 2026)